

Microsoft Foundry Fundamentals

A Complete Guide

13 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:
<https://snehaliteng.github.io/tutorials/foundry-fundamentals/>

1. Introduction

What is Azure AI Foundry?

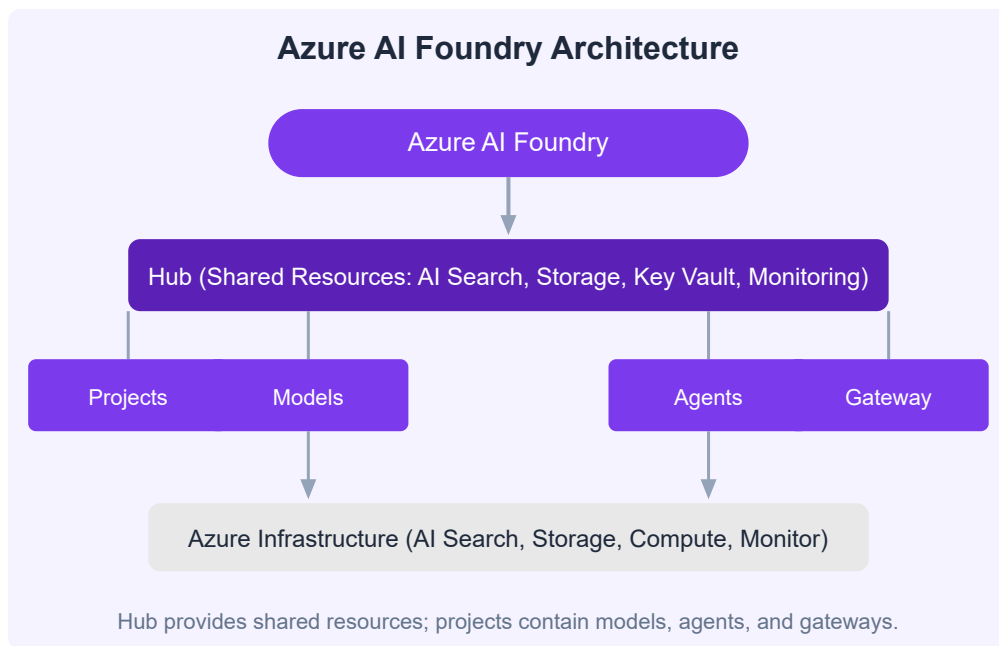
Azure AI Foundry is Microsoft's unified platform for building, deploying, and managing AI applications at scale. It brings together model catalog, agent service, RAG pipelines, API management, and governance under one hub.

Key Capabilities

- **Model Catalog** — Access OpenAI, Meta, Mistral, Cohere, and open-source models
- **Agent Service** — Build and host AI agents with tools, memory, and guardrails
- **AI Search** — Hybrid vector + keyword search for RAG
- **AI Gateway** — Rate limiting, content safety, and MCP support
- **Foundry IQ** — Agentic retrieval with knowledge bases
- **Multi-Modal** — Document Intelligence, PII masking, image analysis

What You Will Build

- AI agents with tools, memory, and MCP servers
- End-to-end RAG pipelines with Azure AI Search
- API gateways with rate limiting and content safety
- Multi-modal apps for document processing
- Multi-agent orchestration and A2A communication



💡 **Key Insight:** Foundry unifies the entire AI lifecycle — from experimentation (model playground) to production (agent service with governance).

✏️ **Exercise:** List three real-world applications you could build with Foundry. For each, identify which capabilities (agent, RAG, gateway, multi-modal) are most relevant.

[← Back to Tutorials](#)

2. Community & Setup

Join the Community

- **Azure AI Foundry Community** — Discussion forums, events, and announcements
- **GitHub** — Sample repos, SDK source, and issue tracking
- **Stack Overflow** — Tag questions with `azure-ai-foundry`

Azure Subscription Setup

1. Create an Azure subscription (free tier works)
2. Navigate to **Azure AI Foundry** in the portal
3. Create a Hub resource (shared AI Search, Storage, Key Vault)
4. Create a Project within the Hub

SDK Installation

```
# Python
pip install azure-ai-projects azure-ai-inference azure-search-documents

# Node.js
npm install @azure/ai-projects @azure/ai-inference @azure/search-documents

# .NET
dotnet add package Azure.AI.Projects
dotnet add package Azure.AI.Inference
```

VSCode Extension

Install the **Azure AI Foundry** extension from the marketplace to browse models, deploy agents, and test RAG pipelines directly from your editor.

 **Note:** The Hub provides shared infrastructure (AI Search, Storage, Key Vault) across all projects. Create one Hub per team.

 **Exercise:** Create a new Azure AI Foundry Hub and Project in your subscription. Install the Python SDK. Verify the setup by running `pip list | findstr azure-ai`.

3. Foundry Primer

Generative AI & Agents

Generative AI models can produce text, code, images, and audio. Agents extend this by adding tool use, memory, and autonomous decision-making. Foundry provides a managed service for both.

SDK Overview

SDK	Package	Purpose
Projects	<code>azure-ai-projects</code>	Agent lifecycle, tools, memory
Inference	<code>azure-ai-inference</code>	LLM chat completions, embeddings
Search	<code>azure-search-documents</code>	Vector + keyword search
Identity	<code>azure-identity</code>	Authentication

Hub vs Standalone Projects

- **Hub** — Shared AI Search, Storage, Key Vault, and monitoring. Best for teams.
- **Standalone Project** — Independent resources. Best for experiments and individual dev.

Deploying Models in Azure

1. Open the Model Catalog in Foundry
2. Select a model (e.g., `gpt-4o` , `Llama-3.1-70b`)

3. Choose deployment type (Global Standard, Provisioned Managed)
4. Configure capacity and content filters
5. Deploy and get the endpoint URL + key

Demo Repositories

Microsoft provides starter repos on GitHub:

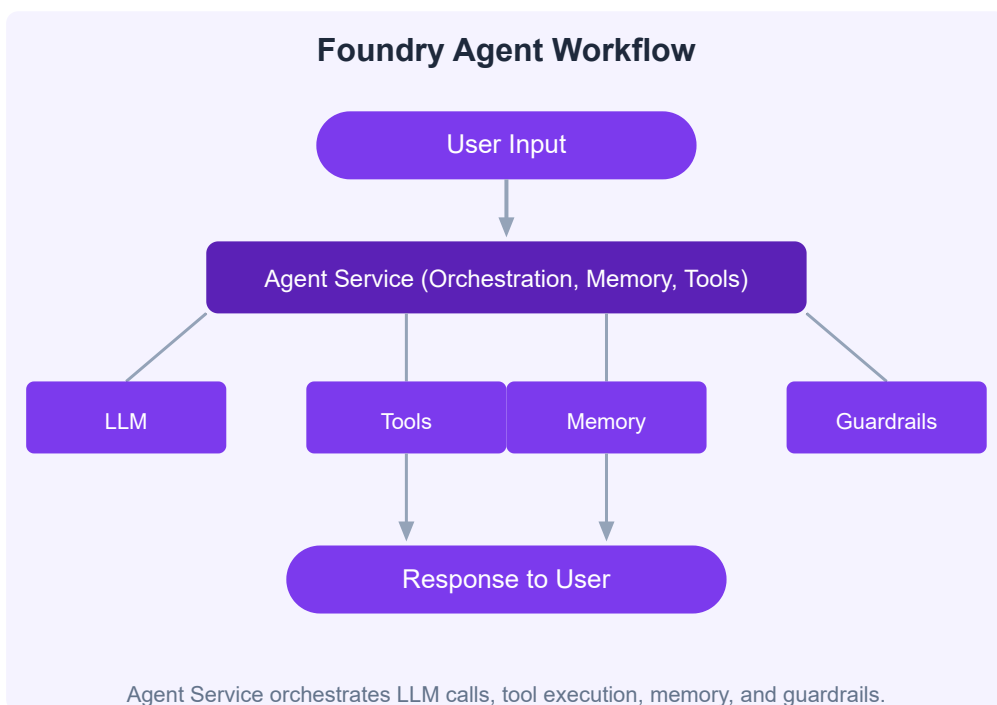
- [Azure-Samples/azure-ai-agent-sdk](#) — Agent SDK samples
- [Azure-Samples/azure-ai-search-rag](#) — RAG pipeline samples
- [Azure-Samples/azure-ai-gateway](#) — AI Gateway samples

 **Exercise:** Deploy a [gpt-4o](#) model from the Foundry model catalog. Note the endpoint URL and authentication method. Write a Python script that sends a test prompt to the endpoint.

4. AI Agents

Agent Service Portal

Foundry's Agent Service provides a managed runtime for AI agents. You can create, configure, and monitor agents through the portal or SDK.



Python Agent Example

```
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential

client = AIProjectClient(
    subscription_id="your-sub",
    resource_group="your-rg",
    project_name="my-project",
    credential=DefaultAzureCredential()
)

agent = client.agents.create_agent(
    model="gpt-4o",
    name="MyFirstAgent",
    instructions="You are a helpful assistant that answers questions about Az
)

response = client.agents.send_message(agent.id, "What is Azure AI Foundry?")
print(response.text)
```

Built-in Tools

Tool	Description
Web Searcher	Search the web for real-time information
Code Interpreter	Execute Python code in a sandboxed environment
OpenAPI Tool	Call any REST API via OpenAPI spec
MCP Server	Connect to MCP-compatible tool servers
Azure AI Search	Query vector indexes for RAG
Memory Store	Persist conversation history and user context

Multi-Tool Agent

```
from azure.ai.projects.models import ToolSet, CodeInterpreterTool, WebSearchTool

toolset = ToolSet()
toolset.add(CodeInterpreterTool())
toolset.add(WebSearchTool())

agent = client.agents.create_agent(
    model="gpt-4o",
    name="MultiToolAgent",
    instructions="Use tools to answer user questions.",
    toolset=toolset
)
```

Browser Automation

Foundry agents can control browser-based workflows for web scraping, form filling, and testing using Playwright-based automation tools.

Memory Store Concepts

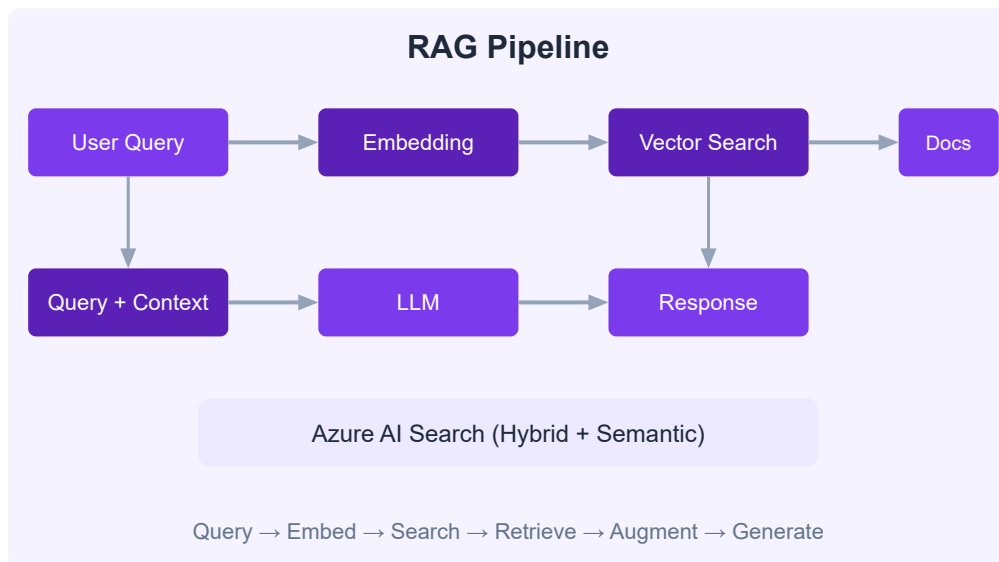
- **Volatile** — In-memory, session-scoped
- **File-based** — Local persistence for development
- **Cosmos DB** — Production-grade, distributed memory

 **Exercise:** Create an agent with both Web Search and Code Interpreter tools. Ask it to "Search for the latest AI news and summarize the top 3 stories." Verify it uses both tools correctly.

5. End-to-End RAG

RAG Basics

Retrieval-Augmented Generation (RAG) combines information retrieval with LLM generation. The model queries a knowledge base, retrieves relevant chunks, and generates answers grounded in those chunks.



Vector Embeddings

Documents are split into chunks, each chunk is converted to a vector embedding using an embedding model, and stored in a vector index.

```
from azure.ai.inference import EmbeddingsClient

embeddings = EmbeddingsClient(endpoint="...", credential=cred)
response = embeddings.embed(
    model="text-embedding-3-small",
    input=["Azure AI Foundry is a unified AI platform."]
)
vector = response.data[0].embedding
```

Azure AI Search

```
from azure.search.documents import SearchClient
from azure.search.documents.indexes import SearchIndexClient

# Create index
index_client = SearchIndexClient(endpoint, credential)
index_client.create_index(index_definition)

# Search with hybrid + semantic
search_client = SearchClient(endpoint, "docs-index", credential)
results = search_client.search(
    search_text="What is Foundry?",
    query_type="semantic",
    vector_queries=[{ "vector": query_vector, "fields": ["content_vector"] }]
    top=5
)
```

Multi-Modal RAG

Extend RAG to handle images, tables, and PDFs. Use Document Intelligence to extract content from documents before chunking and indexing.

Deploying Infrastructure

Use Bicep or Terraform to provision:

- Azure AI Search service
- Storage account for document upload
- AI Services for Document Intelligence
- Foundry Hub and Project

 **Exercise:** Create an Azure AI Search index from a set of PDF documents. Implement a RAG agent that queries this index and generates answers grounded in the documents. Test with questions that require specific document references.

6. API Management & AI Gateway

AI Gateway Concepts

The AI Gateway sits between clients and AI models/agents, providing rate limiting, content safety, load balancing, and observability — all through Azure API Management.

MCP Deep Dive

Model Context Protocol (MCP) standardizes how agents discover and call tools. Foundry's AI Gateway natively supports MCP, allowing agents to use any MCP-compatible tool server.

```
# MCP Tool Definition (JSON)
{
  "name": "get_weather",
  "description": "Get current weather for a location",
  "inputSchema": {
    "type": "object",
    "properties": {
      "location": { "type": "string" }
    },
    "required": ["location"]
  }
}
```

MCP Server Architecture

An MCP server exposes tools via a standardized endpoint. The AI Gateway translates between agent tool calls and MCP server requests.

Custom MCP Servers

```
# Python MCP Server Example
from mcp.server import Server, NotificationOptions
from mcp.server.models import InitializationOptions

server = Server("weather-server")

@server.list_tools()
async def list_tools():
    return [
        Tool(name="get_weather", description="Get weather",
            inputSchema={"type": "object", "properties": {"location": {"type": "string"}},
        ]

@server.call_tool()
async def call_tool(name, arguments):
    if name == "get_weather":
        return {"temperature": 22, "conditions": "sunny"}
    raise ValueError(f"Unknown tool: {name}")
```

Rate Limiting

Configure rate limiting per user, per API key, or per model deployment to prevent abuse and manage costs.

 **Exercise:** Create an AI Gateway in Azure API Management. Configure a rate limit of 10 requests per minute per key. Add a content safety policy that blocks hate speech. Test with both allowed and blocked inputs.

7. Foundry IQ

Introduction to Foundry IQ

Foundry IQ combines agentic retrieval with intelligent ranking. Instead of simple keyword search, agents autonomously decide which knowledge sources to query and how to rank results.

Agentic Retrieval

Unlike static RAG, agentic retrieval lets the agent:

- Choose which knowledge base to query based on the question
- Rewrite queries for better retrieval
- Combine results from multiple sources
- Decide when to fall back to model knowledge

Ranking Techniques

Technique	Description
Semantic Ranking	Re-rank results by semantic similarity to the query
Hybrid Ranking	Combine vector similarity + keyword BM25 scores
LLM Re-ranking	Use an LLM to judge relevance of each result
Reciprocal Rank Fusion	Merge ranked lists from multiple retrieval sources

Knowledge Base Setup

```
from azure.ai.projects import AIProjectClient

# Create a knowledge base connection
kb = client.knowledge_bases.create(
    name="company-docs",
    description="Internal company documentation",
    storage_connection_string=conn_str,
    search_endpoint=search_endpoint,
    chunk_size=512,
    overlap=64
)

# Attach to agent
agent = client.agents.create_agent(
    model="gpt-4o",
    name="IQAgent",
    knowledge_bases=[kb.id]
)
```

Creating Agents via Portal & Python

You can create IQ agents either through the Foundry portal (no-code) or programmatically via the SDK (as shown above).

 **Exercise:** Create a knowledge base from a set of internal documents. Build an agent with Foundry IQ that can answer questions across multiple knowledge bases. Test queries that require combining information from two different sources.

[← Back to Tutorials](#)

8. Multi-Modal Applications

Introduction

Foundry supports multi-modal applications that process text, images, PDFs, tables, and audio. Combine vision models, Document Intelligence, and LLMs for rich document processing workflows.

PII Masking

Detect and redact personally identifiable information (PII) from documents and conversations before processing.

```
from azure.ai.contentsafety import ContentSafetyClient

# PII detection and masking
result = client.detect_pii(
    text="My email is john.doe@example.com and phone is 555-1234"
)
# Returns redacted text and PII entities detected
```

Text Redaction

Automatically redact sensitive content from documents using Azure AI Content Safety or custom regex patterns.

Document Intelligence

```
from azure.ai.documentintelligence import DocumentIntelligenceClient

doc_client = DocumentIntelligenceClient(endpoint, credential)
with open("invoice.pdf", "rb") as f:
    poller = doc_client.begin_analyze_document("prebuilt-invoice", f)
    result = poller.result()

for doc in result.documents:
    print(f"Vendor: {doc.fields.get('VendorName').value}")
    print(f"Total: {doc.fields.get('InvoiceTotal').value}")
```

Invoice Analysis

Build an end-to-end pipeline that:

1. Accepts invoice PDFs via upload
2. Extracts fields with Document Intelligence
3. PII-masks sensitive data
4. Stores results in a searchable index
5. Provides a natural language query interface via an agent

 **Note:** Document Intelligence supports pre-built models for invoices, receipts, identity documents, and custom extraction models.

 **Exercise:** Build a multi-modal agent that accepts an uploaded invoice PDF, extracts key fields (vendor, date, total), PII-masks any personal data, and returns a structured summary. Use Document Intelligence + Content Safety.

[← Back to Tutorials](#)

9. Agent Orchestration

Sequential Workflows

Chain agents in sequence where each agent's output becomes the next agent's input. Useful for document processing pipelines.

```
# Sequential workflow pseudocode
# Agent 1: Extract document metadata
# Agent 2: Classify document type
# Agent 3: Generate summary
# Agent 4: Store results

async def process_document(document):
    metadata = await extract_agent.run(f"Extract metadata from: {document}")
    doc_type = await classify_agent.run(f"Classify: {metadata}")
    summary = await summarize_agent.run(f"Summarize: {document}")
    await storage_agent.run(f"Store: {metadata} | {doc_type} | {summary}")
    return summary
```

Human-in-the-Loop (HITL)

For critical decisions, agents can pause and request human approval before proceeding.

```
# HITL pattern
async def approve_expense(amount, reason):
    if amount > 1000:
        # Pause and ask human
        approval = await ask_human(
            f"Approve ${amount} for {reason}?",
            timeout=3600 # Wait up to 1 hour
        )
        if not approval:
            return "Expense rejected by manager"
    return "Expense approved"
```

Agent Chat Systems

Build multi-turn chat systems where different agents handle different parts of the conversation:

- **Triage Agent** — Routes to the right specialist
- **Support Agent** — Answers technical questions
- **Billing Agent** — Handles invoices and payments
- **Escalation Agent** — For complex issues requiring human intervention

Orchestration Best Practices

- Define clear handoff criteria between agents
- Pass conversation context explicitly during handoffs
- Set timeout limits for human-in-the-loop steps
- Log all orchestration decisions for audit

 **Exercise:** Build a three-agent sequential workflow: (1) a research agent that gathers information, (2) an analysis agent that processes the information, and (3) a reporting agent that generates a formatted report. Add a human-in-the-loop approval step before the report is finalized.

10. Microsoft Agent Framework

MAF in Foundry

The Microsoft Agent Framework (MAF) is integrated with Foundry to provide a rich SDK for building agents with tools, memory, and structured outputs.

Basic Chat Agent

```
import { Agent, AgentClient } from "@microsoft/agents";

const agent = new Agent({
  name: "HelpDeskAgent",
  instructions: "You are an IT help desk assistant.",
  model: "gpt-4o",
});

const client = new AgentClient(agent);
const response = await client.sendMessage("How do I reset my password?");
console.log(response.text);
```

MCP Tool Agent

```
import { Tool } from "@microsoft/agents";

const ticketTool = new Tool({
  name: "create_ticket",
  description: "Create an IT support ticket",
  parameters: {
    title: { type: "string" },
    priority: { type: "string", enum: ["low", "medium", "high"] }
  },
  handler: async (args) => {
    return await createTicketInServiceNow(args);
  }
});

agent.addTool(ticketTool);
```

Multi-Tool Agent

```
// Combine knowledge base search + ticket creation
agent.addTools([
  knowledgeBaseTool,
  ticketTool,
  calendarTool,
  emailTool
]);
```

DevUI Visualization

MAF's DevUI provides a visual debugger for agent conversations, tool calls, and memory state. Use it during development to inspect agent behavior.

Structured Outputs

```
const agent = new Agent({
  name: "DataExtractor",
  instructions: "Extract structured data from text.",
  outputSchema: {
    type: "object",
    properties: {
      name: { type: "string" },
      date: { type: "string" },
      amount: { type: "number" }
    },
    required: ["name", "date", "amount"]
  }
});
```

Sequential & Parallel Workflows

MAF supports orchestrating multiple agents in sequence (pipeline) or in parallel (fan-out) using the Orchestrator.

 **Exercise:** Build a multi-tool MAF agent that uses three tools: a knowledge base search tool, a ticket creation tool, and an email notification tool. Test with a scenario where a user asks about a known issue and requests a ticket.

11. Agent-to-Agent Communication

A2A Protocol

The Agent-to-Agent (A2A) protocol enables autonomous communication between agents built with different frameworks, hosted in different environments, and managed by different teams.

Architecture

A2A uses a client-server model:

- **A2A Server** — Exposes an agent's capabilities via a standardized API
- **A2A Client** — Discovers and invokes remote agents
- **Agent Card** — JSON metadata describing agent capabilities, input/output schemas

Hello World A2A Server

```
from a2a import A2AServer, AgentCard

server = A2AServer()

@server.agent_card
def get_card():
    return AgentCard(
        name="WeatherAgent",
        description="Provides weather information",
        url="http://localhost:8000/a2a",
        capabilities=["get_weather"]
    )

@server.handle("get_weather")
async def get_weather(location: str):
    # Call weather API
    return {"temperature": 22, "conditions": "sunny"}

server.run(port=8000)
```

Exposing Agents via A2A

```
# Expose any Foundry agent as an A2A endpoint
from a2a import A2AAdapter
from azure.ai.projects import AIProjectClient

adapter = A2AAdapter(client, agent_id)
adapter.expose(port=8000)
```

Streaming Responses

A2A supports streaming for real-time agent communication:

```
async for chunk in a2a_client.send_message_stream("What's the weather?", agent_id):
    print(chunk.text, end="")
```

MCP + A2A

Combine both protocols: use MCP for tool exposure and A2A for agent-to-agent communication. MCP provides tools; A2A provides agent orchestration across boundaries.

Publishing to Azure

Deploy A2A servers to Azure Container Apps, AKS, or Azure Functions. Use Azure API Management to manage, secure, and monitor A2A endpoints.

 **Exercise:** Create two agents: a Weather Agent (A2A server) and a Travel Agent (A2A client). The Travel Agent should discover and call the Weather Agent to get forecasts for trip planning. Implement streaming responses.

[← Back to Tutorials](#)

12. Practice & Resources

Quick Reference Guide

Task	Command / Code
Create agent	<code>client.agents.create_agent(model, name, instructions)</code>
Add tools	<code>toolset.add(CodeInterpreterTool())</code>
Send message	<code>client.agents.send_message(agent_id, text)</code>
Create search index	<code>index_client.create_index(index_def)</code>
Query search	<code>search_client.search(search_text, vector_queries=...)</code>
Deploy model	Portal → Model Catalog → Deploy

Useful Resources

- [Azure AI Foundry Documentation](#)
- [Azure AI Agent SDK Samples](#)
- [RAG Pipeline Samples](#)
- [Pricing Calculator](#)

Study Plan

Week	Topic	Activities
1	Foundations & Setup	Read Ch 1-3, setup Azure + SDK
2	Agents & Tools	Read Ch 4, build first agent with tools
3	RAG & Search	Read Ch 5, build RAG pipeline
4	Gateway & IQ	Read Ch 6-7, configure gateway
5	Multi-Modal & Orchestration	Read Ch 8-9, multi-modal pipeline
6	MAF & A2A	Read Ch 10-11, A2A communication
7	Capstone Project	Build end-to-end solution

Bootcamp & Certificate Path

- **Microsoft Learn: Azure AI Fundamentals** (AI-900)
- **Microsoft Learn: Azure AI Engineer** (AI-102)
- **Foundry-specific learning paths** on Microsoft Learn

 **Final Capstone:** Build an end-to-end solution that includes: (1) an agent with multiple tools, (2) a RAG knowledge base, (3) an AI Gateway with rate limiting, (4) multi-modal document processing, and (5) agent-to-agent communication between two specialized agents.

13. References

Glossary

Term	Definition
Foundry Hub	Shared resource container (AI Search, Storage, Key Vault)
Foundry Project	Workspace within a Hub for building AI applications
Agent Service	Managed runtime for AI agents with tools and memory
RAG	Retrieval-Augmented Generation — ground LLM responses in retrieved data
MCP	Model Context Protocol — standardized tool interface
A2A	Agent-to-Agent protocol for inter-agent communication
AI Gateway	API Management layer for AI models and agents
Foundry IQ	Agentic retrieval with intelligent ranking

Cheatsheet

```
# Python SDK Quick Reference

# 1. Create project client
from azure.ai.projects import AIProjectClient
client = AIProjectClient(subscription_id, resource_group, project_name, credential)

# 2. Create agent
agent = client.agents.create_agent(model="gpt-4o", name="MyAgent", instructions="")

# 3. Add tools
from azure.ai.projects.models import ToolSet, CodeInterpreterTool, WebSearchTool
toolset = ToolSet()
toolset.add(CodeInterpreterTool())

# 4. Send message
response = client.agents.send_message(agent.id, "Hello")

# 5. Create search index
from azure.search.documents.indexes import SearchIndexClient
index_client = SearchIndexClient(endpoint, credential)
index_client.create_index(index_definition)

# 6. Deploy model (portal or CLI)
# az ml model create --name gpt-4o --type deployment
```

API Reference

- `Azure.AI.Projects` — Agent lifecycle, tools, memory
- `Azure.AI.Inference` — Chat completions, embeddings
- `Azure.Search.Documents` — Vector + keyword search
- `Azure.Identity` — `DefaultAzureCredential`, `ManagedIdentity`
- `Azure.AI.ContentSafety` — PII detection, content filtering
- `Azure.AI.DocumentIntelligence` — Document extraction

Continue Your Journey

- Build a production agent with monitoring and guardrails

- Contribute to the Foundry SDK on GitHub
- Join the Azure AI community discussions
- Explore the MAF Fundamentals guide for deeper agent-building skills

 **Congratulations!** You've completed the Microsoft Foundry Fundamentals guide. You now have the knowledge to build, deploy, and manage AI agents at scale using Azure AI Foundry.